Now that we have a Dist-DQN and a way to generate target distributions, we need a loss function that will calculate how different the predicted action-value distribution is from the target distribution; then we can backpropagate and do gradient descent as usual, to update the Dist-DQN parameters to be more accurate next time. We often use the mean squared error (MSE) loss function when trying to minimize the distance between two batches of scalars or vectors, but this is not an appropriate loss function between two probability distributions. But there are many possible choices for a loss function between probability distributions. We want a function that will measure how different or distant two probability distributions are and that will minimize that distance.

In machine learning, we are usually trying to train a parametric model (e.g., a neural network) to predict or produce data that closely matches empirical data from some data set. Thinking probabilistically, we can conceive of a neural network as generating synthetic data and trying to train the neural network to produce more and more realistic data—data that closely resembles some empirical data set. This is how we train *generative* models (models that generate data); we update their parameters so that the data they generate looks very close to some training (empirical) data set.

For example, let's say we want to build a generative model that produces images of celebrities' faces (figure 7.16). In order to do this, we need some training data, so we use the freely available CelebA data set that contains hundreds of thousands of high quality photographs of various celebrities such as Will Smith and Britney Spears. Let's call our generative model P and this empirical data set Q.

Figure 7.16. A generative model can be a probabilistic model that trains by maximizing the probability that it generates samples that are similar to some empirical data set. Training happens in an iterative loop where the empirical data is supplied to the generative model, which tries to maximize the probability of the empirical data. Before training, the generative model will assign low probability to examples taken from the training data set, and the objective is for the generative model to assign high probability to examples drawn from the data set. After a sufficient number of iterations, the generative model will have assigned high probability to the empirical data, and we can then sample from this distribution to generate new, synthetic data.